UTM

UNIVERSITI TEKNOLOGI MALAYSIA

FACULTY OF COMPUTING

SEMESTER 2 2024/2025

SECP2753 DATA MINING

SECTION 02

Project 1

Lecturer: Dr. Rozilawati Binti Dollah @ Md. Zain

Student Name	Matric No.
LAM YOKE YU	A23CS0233
TAN YI YA	A23CS0187
TAN ZHI MING	A23CS0189

Introduction

This report documents the process and findings of analysing the “Spotify Tracks Dataset” from Kaggle. The dataset stores Spotify songs with 114 genres and their audio features. 3 different classification algorithms, namely the Support Vector Machine (SVM), the decision tree and the random forest, are applied to several specific business insights.

In terms of implementation, we decided to use Notebook instead of Data Mining tools. Due to the limitation of our knowledge, Data Mining tools such as KNIME are not as intuitive. On the other hand, notebooks give us more control over the process, allowing us to clearly see what is going on in each step.

Data Preparation

After loading the dataset, we checked if there are empty values in the dataset.

```
df = pd.read_csv('dataset.csv')
df.isnull().sum()
```

It is found that there is only one empty value in `artists`, `album_name` and `track_name` respectively. Since these attributes do not make significance in the analysis, we can just replace them with arbitrary strings without dropping any rows or columns.

```
df['artists'] = df['artists'].fillna('Unknown Artist')
df['album_name'] = df['album_name'].fillna('Unknown Album')
df['track_name'] = df['track_name'].fillna('Unknown Track')
```

Model Development

For each business insight, we need to identify its parameters and target variables.

Business Insight 1: Predicting Hit Songs Using Popularity Correlations

Target Variable: The tracks are classified as “hit” or “non-hit” based on a popularity threshold
Parameters: `danceability`, `energy`, `loudness`, `acousticness`, `instrumentalness`, `liveness`, `tempo`

The top 25% tracks with the highest popularity will be categorised as “hit songs”. A new binary column, “`hit_song`” is created.

```
threshold = df['popularity'].quantile(0.75)
df['hit_song'] = df['popularity'].apply(lambda x: 1 if x >
threshold else 0)
```

Then, the target variables and parameters are stored in different dataframe and scaled.

```
X = df[['danceability', 'energy', 'loudness', 'acousticness',
'instrumentalness', 'liveness', 'tempo']]
y = df['hit_song']
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

The dataset is split into a training set and testing set with a 70:30 ratio.

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

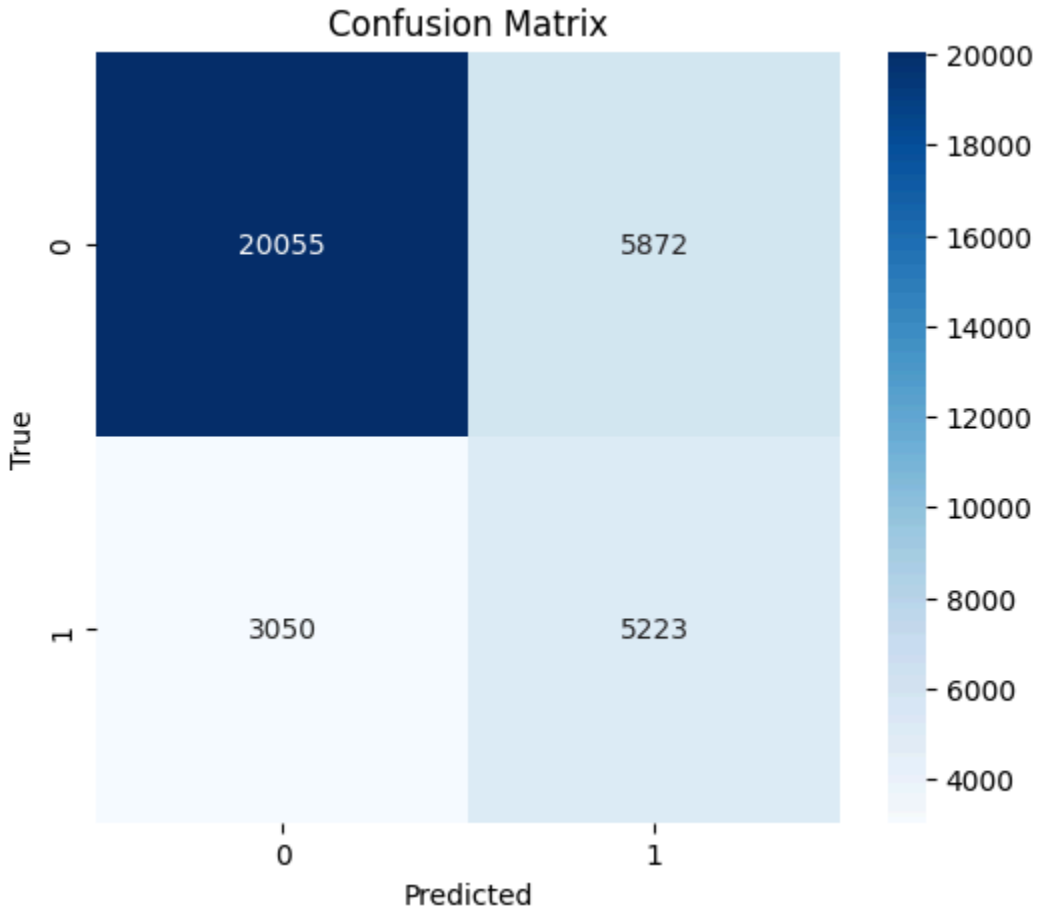
Since the hit songs are the top 25% tracks with the highest popularity, the class is unbalanced and thus the training set has to be resampled.

```
sm = SMOTE(random_state=42)
X_train_res, y_train_res = sm.fit_resample(X_train, y_train)
```

The data is fitted into different models and a confusion matrix is generated for better visualisation.

Decision Tree

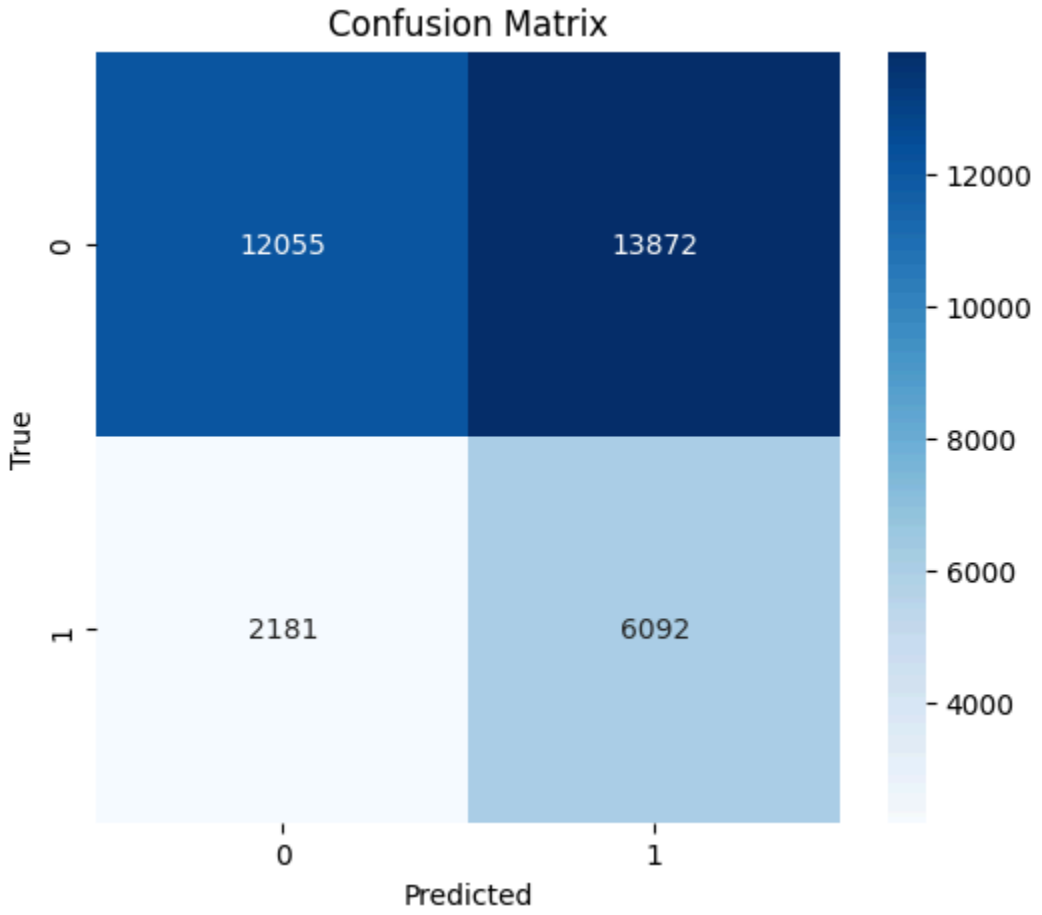
```
model_dt = DecisionTreeClassifier(random_state=42)
model_dt.fit(X_train_res, y_train_res)
y_pred_dt = model_dt.predict(X_test)
```



		Actual Value	
		Positives (Hit Songs)	Negatives (Non-Hit)
Predicted Value	Positives (Hit)	5223	3050
	Negatives (Non-Hit)	5872	20055

SVM

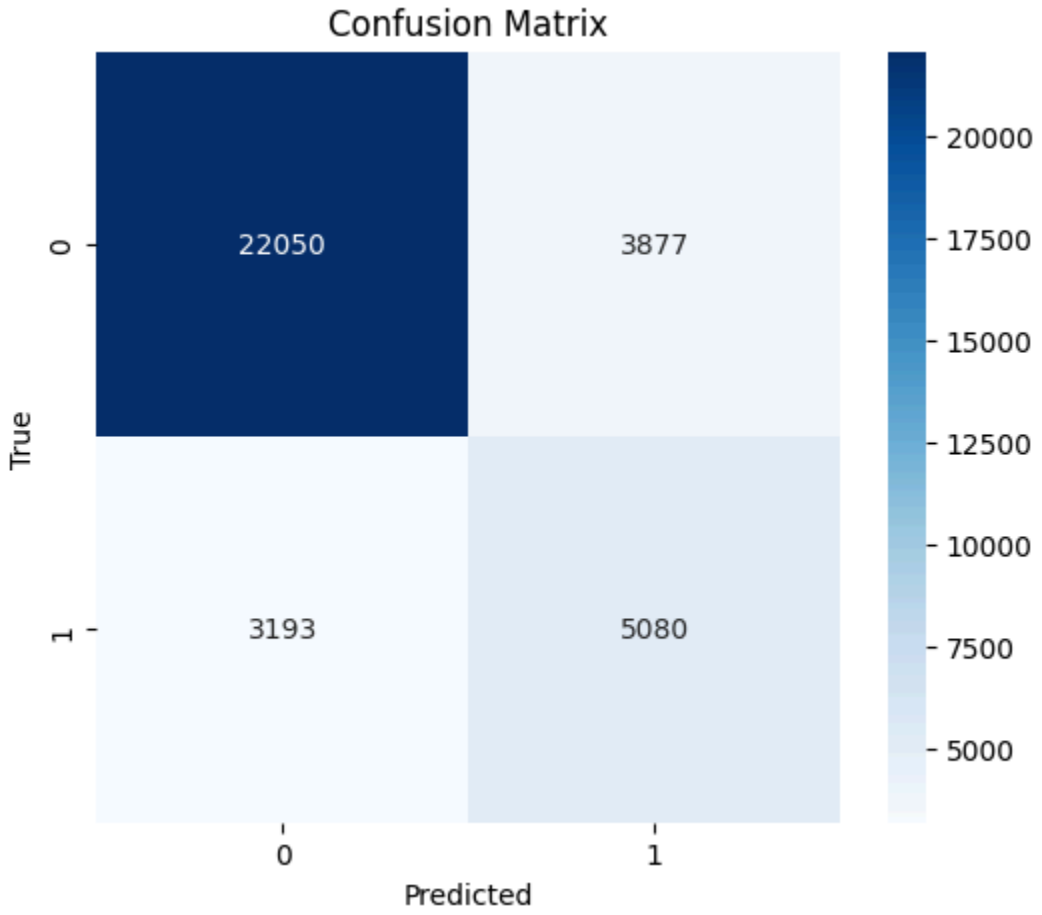
```
model_svm = SVC()
model_svm.fit(X_train_res, y_train_res)
y_pred_svm = model_svm.predict(X_test)
```



		Actual Value	
		Positives (Hit Songs)	Negatives (Non-Hit)
Predicted Value	Positives (Hit)	6092	2181
	Negatives (Non-Hit)	13872	12055

Random Forest

```
model_rf = RandomForestClassifier(random_state=42)
model_rf.fit(X_train_res, y_train_res)
y_pred_rf = model_rf.predict(X_test)
```



		Actual Value	
		Positives (Hit Songs)	Negatives (Non-Hit)
Predicted Value	Positives (Hit)	5080	3193
	Negatives (Non-Hit)	3877	22050

Business Insight 2: Predicting Music Danceability Using Audio Features and Genre Classification

Target Variable: The tracks are classified into three danceability levels based on their continuous danceability score:

- **Low Danceability (0)**: 0.00–0.33
- **Medium Danceability (1)**: 0.34–0.66
- **High Danceability (2)**: 0.67–1.00

These categories are defined using binning. Any edge case that contains Nan values is removed. Then, data type of danceability class are converted into integers for classification.

```
bins = [0.0, 0.33, 0.66, 1.0]
labels = [0, 1, 2]
df['danceability_class'] = pd.cut(df['danceability'], bins=bins,
labels=labels, include_lowest=True)
df = df.dropna(subset=['danceability_class'])
df['danceability_class'] = df['danceability_class'].astype(int)
```

The model is trained on the following acoustic features: `energy, loudness, acousticness, instrumentalness, liveness, tempo, valence, track_genre`

`track_genre` which is an object-type column is label_encoded. Then, any missing value are filled with column mean.

```
for col in X.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    X[col] = le.fit_transform(X[col])
X = X.fillna(X.mean())
```

The target label '`danceability_class`' is removed from X and stored in y. The original '`danceability`' feature is also removed from X to avoid data leakage.

```
X = df.drop(columns=['danceability', 'danceability_class'])
y = df['danceability_class']
```

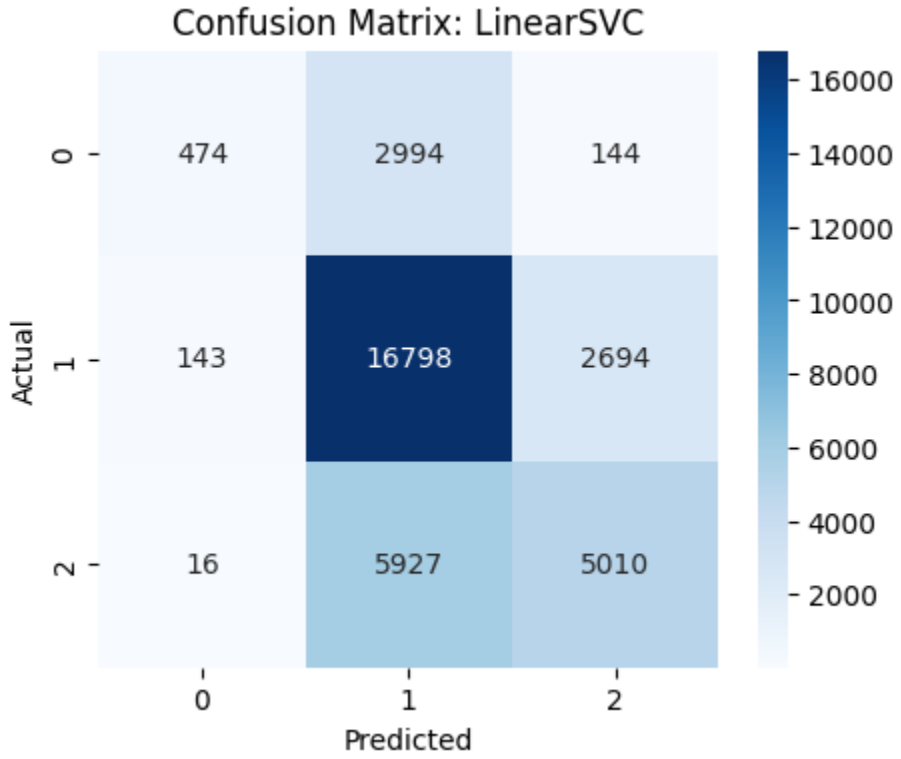
Features in X are scaled with StandardScaler so that each has a mean of 0 and standard deviation of 1 and data is split into 70% training and 30% testing.

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y,
test_size=0.3, random_state=42)
```

Three classification models were trained and a confusion matrix was generated for each of them.

SVM

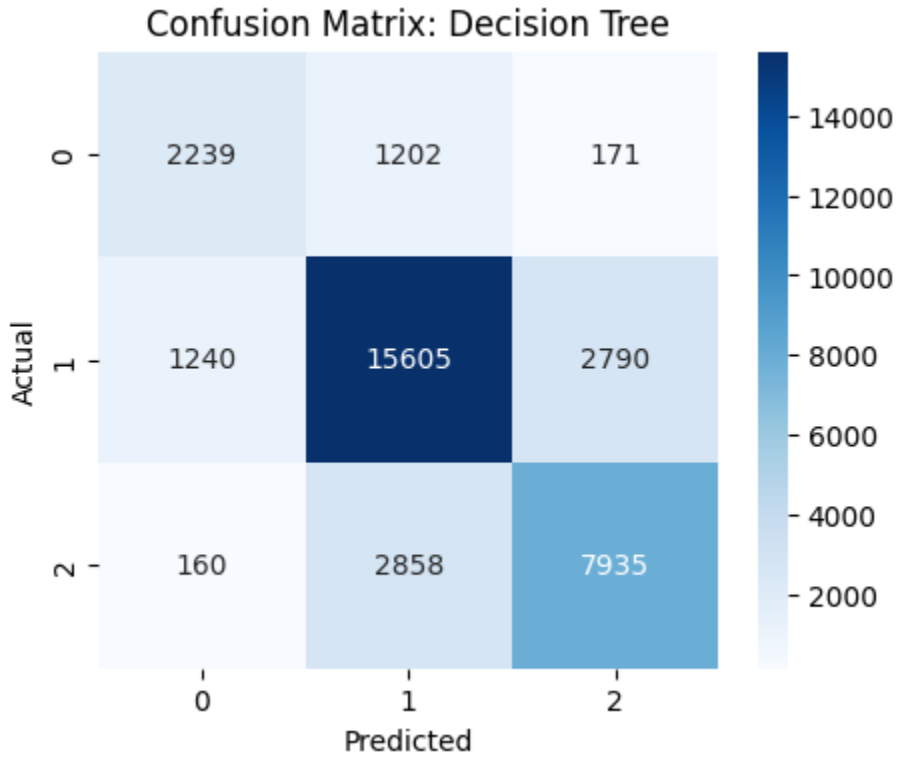
```
svm_model = LinearSVC(max_iter=10000)
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
```



		Predicted Value		
		0 (Low Danceability)	1 (Medium Danceability)	2 (High Danceability)
Actual Value	0 (Low Danceability)	474	2994	144
	1 (Medium Danceability)	143	16798	2694
	2 (High Danceability)	16	5927	5010

Decision Tree

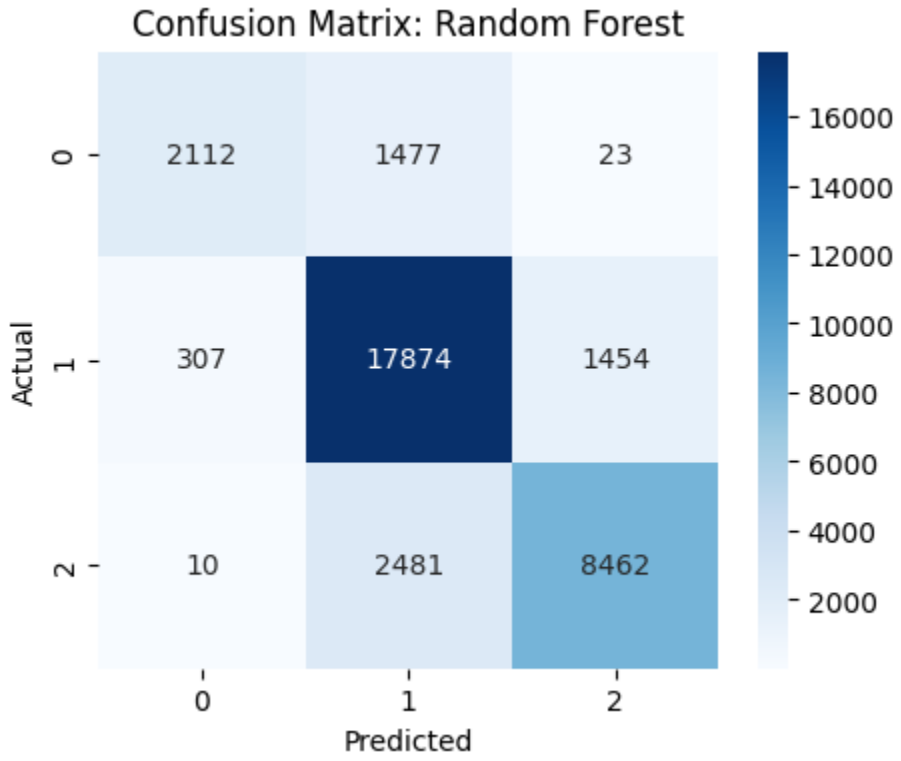
```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
```



		Predicted Value		
		0 (Low Danceability)	1 (Medium Danceability)	2 (High Danceability)
Actual Value	0 (Low Danceability)	2239	1202	171
	1 (Medium Danceability)	1240	15605	2790
	2 (High Danceability)	160	2858	7935

Random Forest

```
rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
```



		Predicted Value		
		0 (Low Danceability)	1 (Medium Danceability)	2 (High Danceability)
Actual Value	0 (Low Danceability)	2112	1477	23
	1 (Medium Danceability)	307	17874	1454
	2 (High Danceability)	10	2481	8462

Business Insight 3: Classifying Songs by Instrumentation Level Using Audio Features

Target Variable: instrument_class. This is created using pd.qcut() on the instrumentalness column to group songs into 3 equally sized categories:

Class	Meaning
0	*Vocal-heavy*
1	*Balanced*
2	*Instrumental*

The model is trained on the following acoustic features: `['acousticness', 'speechiness', 'loudness', 'energy', 'valence', 'tempo', 'duration_ms']`

```
df = df[[
    'instrumentalness', 'acousticness', 'speechiness',
    'loudness', 'energy', 'valence', 'tempo', 'duration_ms'
]]
df = df.dropna()
df = df.drop_duplicates()
```

Below code categorizes songs into three instrumentation levels (Vocal-heavy, Balanced, Instrumental) based on instrumentalness using quantile binning, removes any rows without a class, and converts the labels to integers for model training.

```
df['instrument_class'] = pd.qcut(df['instrumentalness'], q=3,
labels=[0, 1, 2])
df = df.dropna(subset=['instrument_class'])
df['instrument_class'] = df['instrument_class'].astype(int)
```

Selects seven audio features as input (X) and sets the target variable (y) as the instrumentation class. Then, it standardizes the input features using StandardScaler to ensure all features are on the same scale for better model performance.

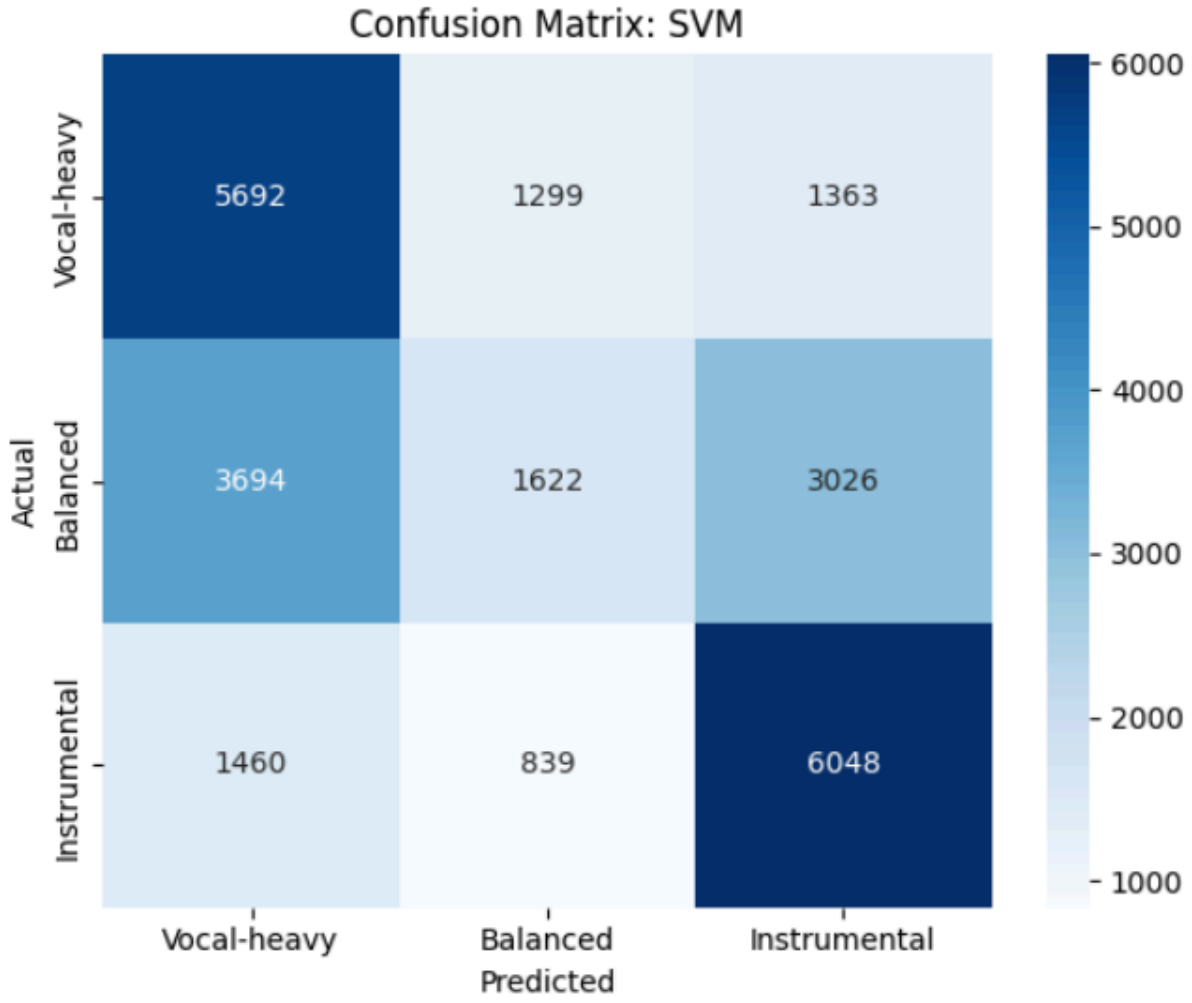
```
X = df[[
    'acousticness', 'speechiness', 'loudness',
    'energy', 'valence', 'tempo', 'duration_ms'
]]
y = df['instrument_class']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

Below code splits the dataset into 70% training and 30% testing sets while maintaining the class distribution (stratify=y) to ensure balanced representation in both sets.

```
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)
```

SVM

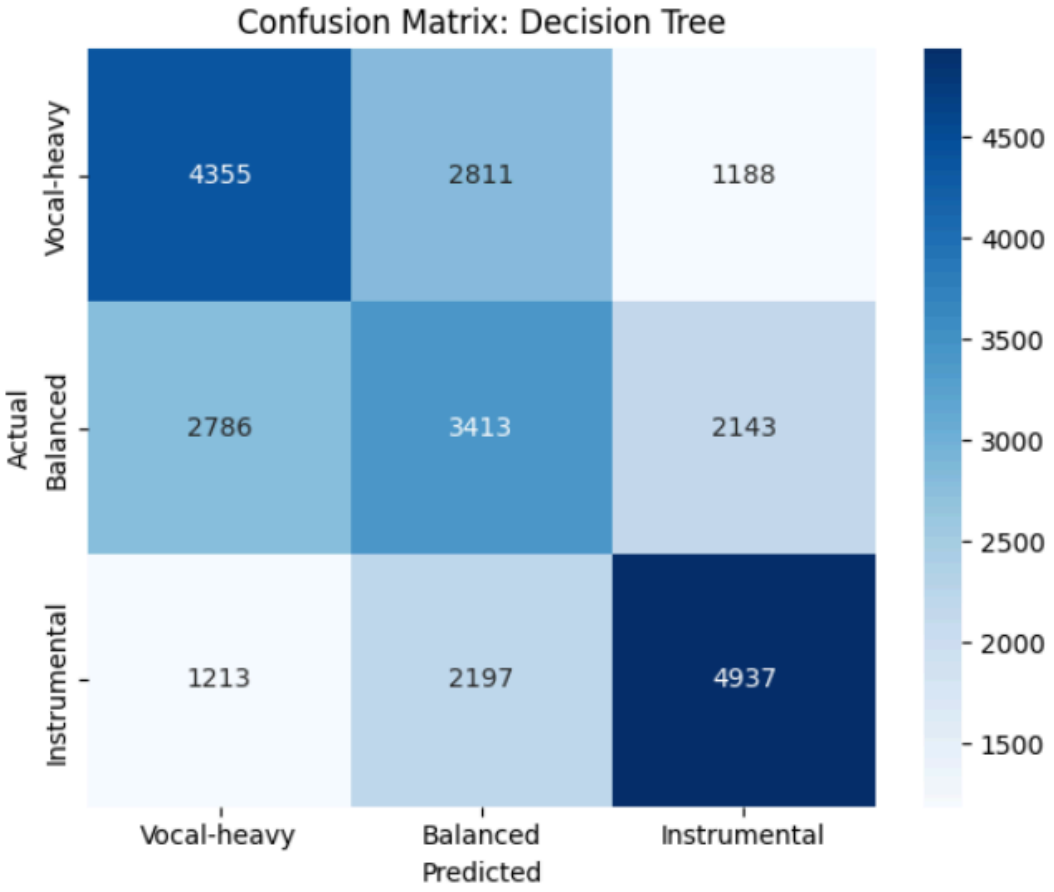
```
svm_model = LinearSVC(max_iter=10000)
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)
```



		Predicted Value		
		0 (Vocal-heavy)	1 (Balanced)	2 (Instrumental)
Actual Value	0 (Vocal-heavy)	5692	1299	1363
	1 (Balanced)	3694	1622	3026
	2 (Instrumental)	1460	382839	6048

Decision Tree

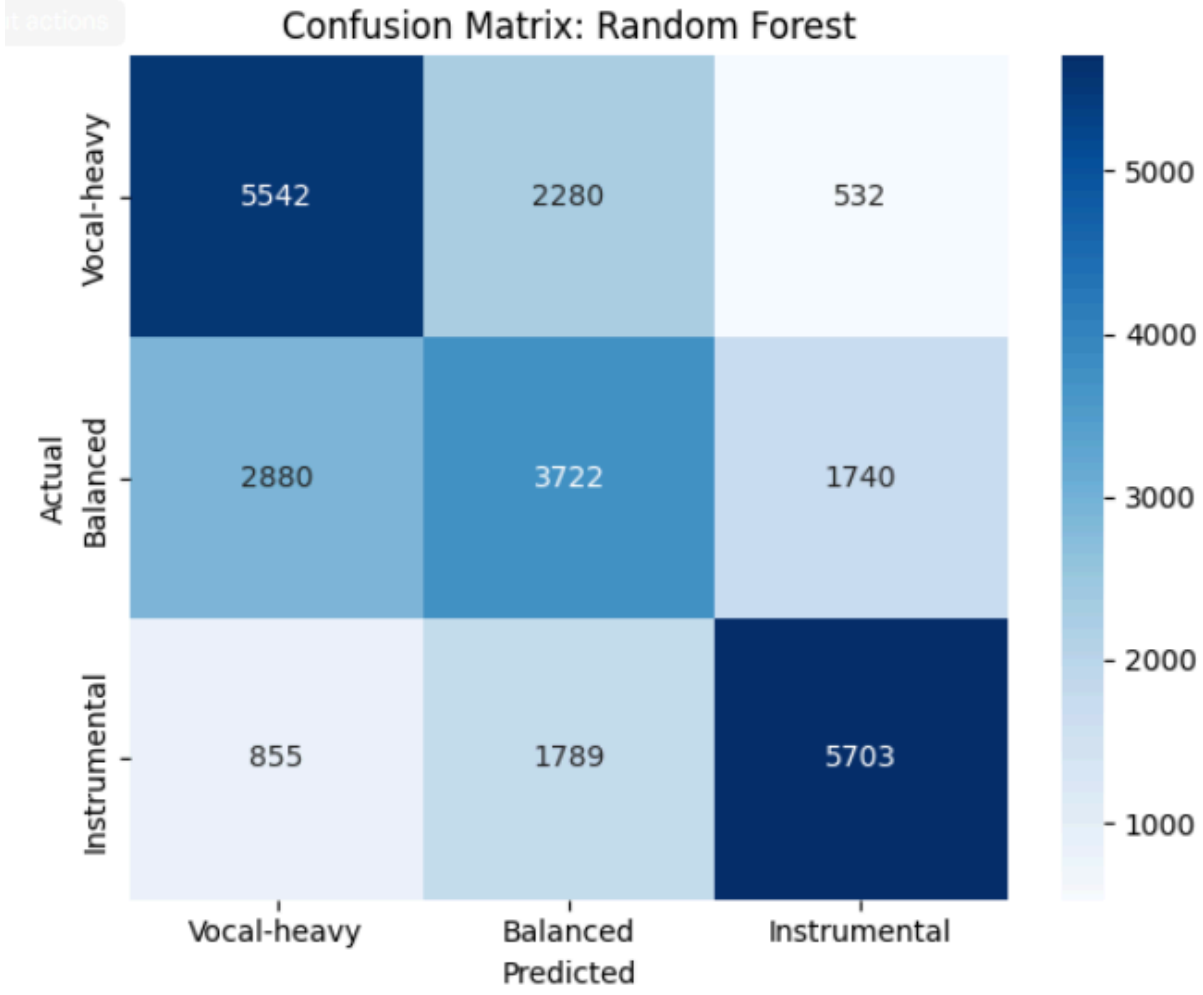
```
dt_model = DecisionTreeClassifier(random_state=42)
dt_model.fit(X_train, y_train)
y_pred_dt = dt_model.predict(X_test)
```



		Predicted Value		
		0 (Vocal-heavy)	1 (Balanced)	2 (Instrumental)
Actual Value	0 (Vocal-heavy)	4355	2811	1188
	1 (Balanced)	2786	3413	2143
	2 (Instrumental)	1213	2197	4937

Random Forest

```
rf_model = RandomForestClassifier(n_estimators=100,
random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)
```



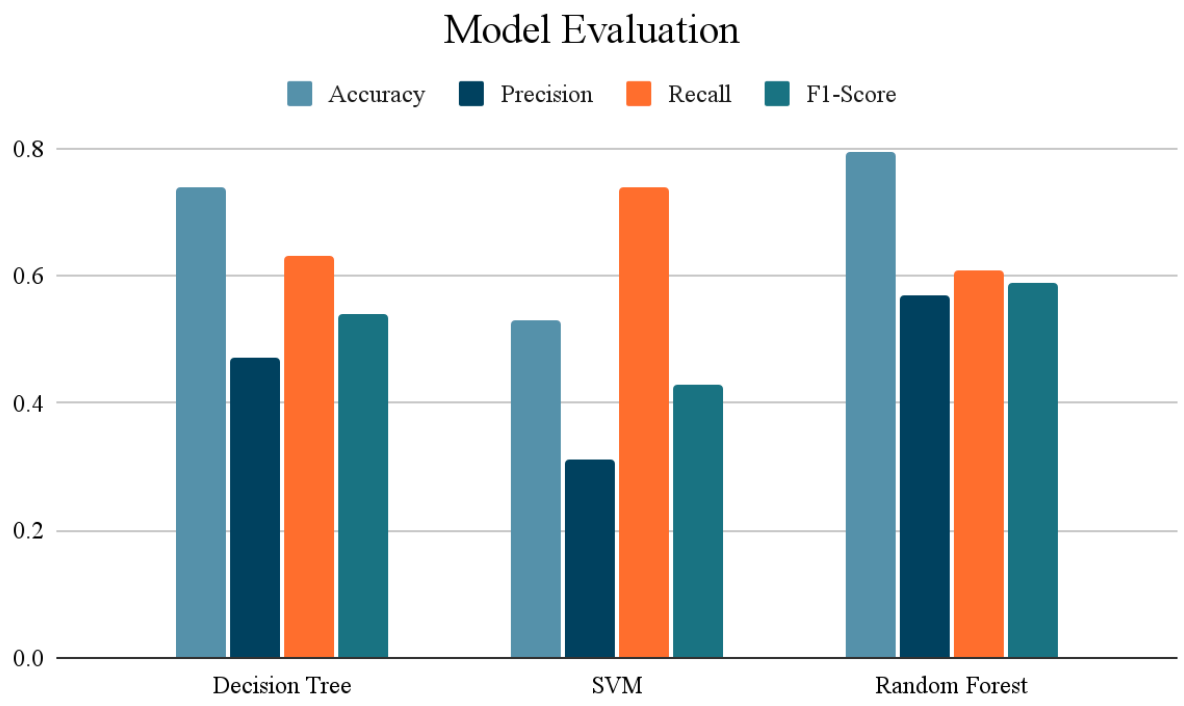
		Predicted Value		
		0 (Vocal-heavy)	1 (Balanced)	2 (Instrumental)
Actual Value	0 (Vocal-heavy)	5542	2280	532
	1 (Balanced)	2880	3722	1740
	2 (Instrumental)	855	1789	5703

Model Evaluation

Each model is evaluated with its Accuracy, Precision, Recall, and F1-score.

Business Insight 1: Predicting Hit Songs Using Popularity Correlations

	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.7391	0.47	0.63	0.54
SVM	0.5306	0.31	0.74	0.43
Random Forest	0.7933	0.57	0.61	0.59



Based on the evaluation, we can see that both the Decision Tree and Random Forest models achieve relatively high accuracy (greater than 0.70).

In terms of precision, when the Random Forest model predicts a song as a hit, it is correct 57% of the time. Despite having the highest precision among all models, it is more conservative in categorising songs as hits, which is indicated by its lower recall. However, the model still incorrectly labels almost half of the non-hit songs as hits.

On the other hand, the SVM model achieves the highest recall at 0.74, meaning it correctly identified 74% of the actual hit song. However, it suffers from low accuracy and precision, indicating a high number of

false positives. With its highest recall, the SVM model could be useful in cases where capturing as many hit songs as possible is the priority.

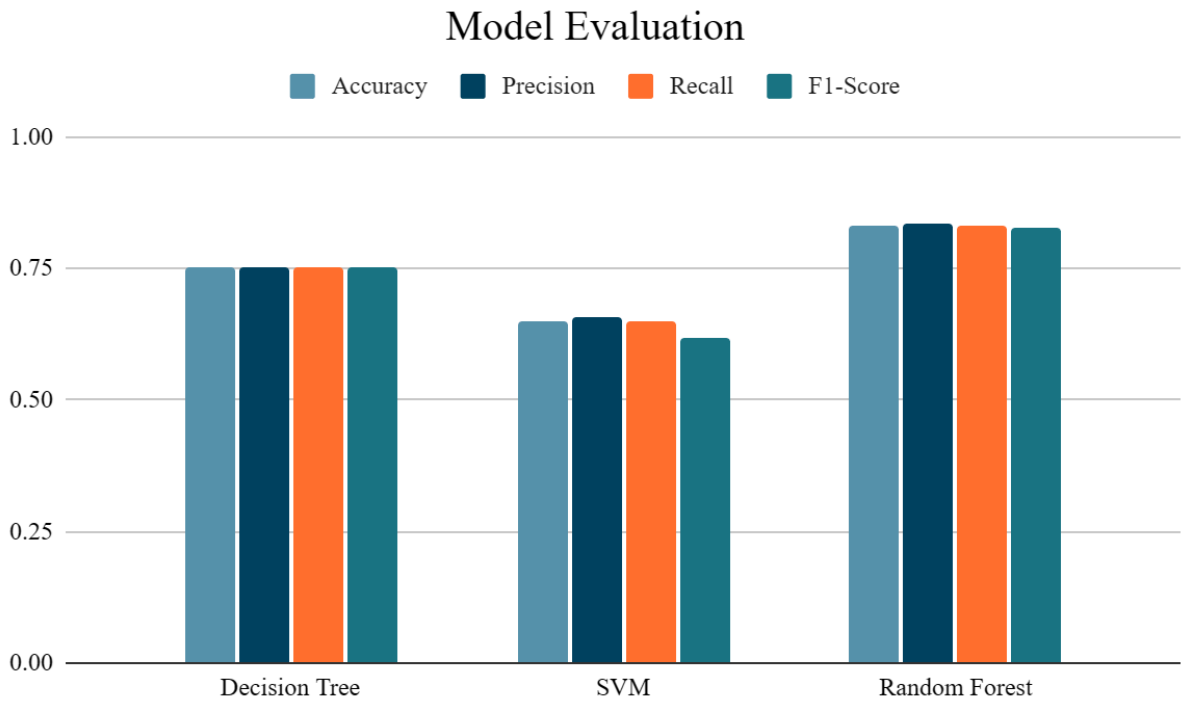
The Decision Tree model has a precision of 47%, meaning that when it predicts a hit, it is correct less than half the time. However, it achieves a recall of 0.63, which means it misses 37% of true hits.

Based on the F1-score, we can deduce that both Decision Tree and Random Forest are just slightly above average, with Random Forest performing slightly better, indicating more reliable and balanced classification performance.

In conclusion, the Random Forest model performs the best, with the highest accuracy and F1-Score, while the SVM model performs the worst, as its overall accuracy is low, despite its strong recall.

Business Insight 2: Predicting Music Danceability Using Audio Features and Genre Classification

	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.7538	0.7538	0.7538	0.7538
SVM	0.6515	0.6585	0.6515	0.6196
Random Forest	0.8318	0.8345	0.8318	0.8282



The above table and chart shows the model evaluation across three models by 4 evaluation metrics which are Accuracy, Precision, Recall and F1-Score. All of the models above show moderate accuracy, with Decision Tree and Random Forest having an accuracy of more than 75%, while random forest with the accuracy of 65%.

For Precision, Random Forest had the highest precision at 83.45%, meaning its predictions were highly accurate. Decision Tree matched its overall accuracy with a precision of 75.38%, while the SVM with 65.85%, indicating a higher chance of false positives.

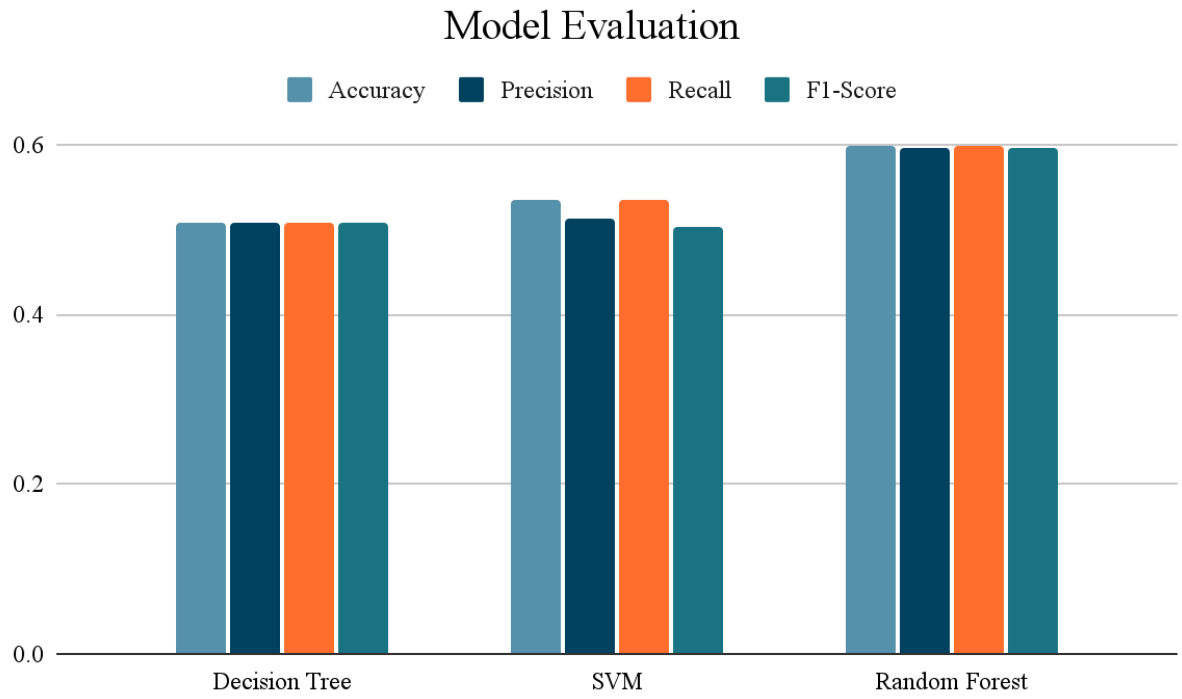
Looking at Recall, Random Forest had the highest Recall value of 83.18%, showing strong detection ability. The Decision Tree achieved 75.38%, and the SVM had the lowest recall at 65.15%, suggesting it missed a greater number of actual cases.

Random Forest had the highest F1-score at 82.82%, indicating it maintained both strong precision and recall. Decision Tree scored 75.38%, showing balanced but lower performance while SVM model had the lowest F1-score of 61.96%.

In conclusion, Random Forest is the best-performing model across all evaluation metrics, making it the most suitable for predicting danceability classes in this dataset. Decision Tree offers consistent but moderately lower performance, and SVM falls short across all metrics, indicating it may need further tuning or is less suited for this specific task.

Business Insight 3: Predicting the type of track based on energy and acousticness

	Accuracy	Precision	Recall	F1-Score
Decision Tree	0.5073	0.5079	0.5073	0.5076
SVM	0.5336	0.5119	0.5335	0.5016
Random Forest	0.5977	0.5967	0.5976	0.5963



Based on the evaluation metrics, random forests outperform both SVM and Decision Tree in the classifying track types based on energy and acousticness. It achieved the highest accuracy rate (0.5977), precision (0.5967), recall rate (0.5976), and f1 score (0.5963), indicating strong and consistent performance across all evaluation dimensions. This indicates that Random Forest can better capture the potential patterns in the dataset, possibly because its integration method combines multiple trees to reduce variance and improve generalization.

Although the recall rate of the SVM model is slightly higher than that of the Decision Tree (0.5335 vs 0.5073), its f1 score is slightly lower (0.5016), highlighting its weak balance between accuracy and recall rate. The overall performance of the decision tree model is the lowest, with all indicators hovering around 0.507. This indicates that it may not be robust enough to capture the more subtle differences between orbital types.

To sum up, the random forest is the most suitable model for this classification task. It offers better predictive ability and generalization, with no signs of overfitting, making it the preferred choice for music classification by instrument level in the real world.

Conclusion

Across all three business insights, the Random Forest model outperformed both the Decision Tree and SVM model. This shows that it has stronger predictive power and is more reliable compared to the other two models.